Day 13 DSA Track — Heap / Priority Queue

1. Beginner-Friendly Summary

A **heap** is useful when you repeatedly need the **smallest or largest item without sorting everything**.

The three recognition patterns for today are:

```
Top-K
    → Keep only the best K candidates.

Streaming
    → Data arrives continuously.
    → Cannot repeatedly sort everything seen so far.

Scheduling
    → Always process the next task according to
      priority / deadline / finish time.
```

Python's `heapq` implements a **min-heap**:

```
import heapq

heap = []

heapq.heappush(heap, 10)
heapq.heappush(heap, 4)
heapq.heappush(heap, 7)

print(heapq.heappop(heap))
```

Output:

```
4
```

The most important invariant is:

> `heap[0]` is always the smallest element in a Python min-heap.

For **Top K largest** problems, an especially useful technique is:

```
Maintain a MIN heap of size K.

heap[0]
   ↓
smallest among the current K best elements
   ↓
candidate must beat this element to enter Top-K
```

---

2. Heap vs Priority Queue

These terms are related but slightly different.

A **priority queue** is an abstract data structure:

```
insert(item, priority)
remove_highest_priority()
peek_highest_priority()
```

A **heap** is one common implementation of a priority queue.

Conceptually:

```
Priority Queue
      |
      | implementation
      v
     Heap
```

Other structures could theoretically implement a priority queue, but heaps provide an excellent balance:

```
Insert     O(log n)
Pop        O(log n)
Peek       O(1)
Build heap O(n)
```

---

3. Heap Structure

A binary min-heap obeys:

```
Parent <= Children
```

Example:

```
          2
        /   \
       5     4
      / \   / \
     9   8 7   6
```

Notice something important:

This is **not completely sorted**.

For example:

```
5 > 4
```

even though `5` appears earlier visually.

A heap guarantees only the parent-child relationship necessary to efficiently retrieve the minimum.

---

4. Array Representation

Heaps are usually represented using arrays.

For index `i`:

```
left child  = 2*i + 1
right child = 2*i + 2
parent      = (i-1) // 2
```

Example:

```
Heap:

          2
        /   \
       5     4
      / \
     9   8


Array:

index:  0  1  2  3  4
value: [2, 5, 4, 9, 8]
```

No tree nodes are required.

---

5. Python Heap Operations

```
import heapq
```

Push

```
heapq.heappush(heap, value)
```

Complexity:

```
O(log n)
```

Pop minimum

```
value = heapq.heappop(heap)
```

Complexity:

```
O(log n)
```

Peek minimum

```
heap[0]
```

Complexity:

```
O(1)
```

Convert list into heap

```
heapq.heapify(values)
```

Complexity:

```
O(n)
```

Not:

```
O(n log n)
```

`heapify()` uses a bottom-up construction algorithm.

Push and immediately pop

```
heapq.heappushpop(heap, value)
```

Useful for bounded heaps.

---

6. Max Heap in Python

Traditionally, Python `heapq` is used as a min-heap.

A common way to simulate a max-heap is storing negative values:

```
numbers = [5, 2, 9, 4]

heap = []
```

```
for x in numbers:
    heapq.heappush(heap, -x)

largest = -heapq.heappop(heap)

print(largest)
```

Output:

```
9
```

Conceptually:

```
Original:

9 > 5 > 4 > 2


Negated:

-9 < -5 < -4 < -2
```

Therefore Python's minimum becomes our original maximum.

---

7. Recognition Signals

Before coding, recognize when a heap is appropriate.

Signal 1 — "Top K"

Words such as:

```
K largest
K smallest
K closest
K most frequent
K highest scoring
K cheapest
```

should immediately make you consider a heap.

Example:

```
Find the 10 largest numbers among 10 million values.
```

Sorting 10 million values may be unnecessary.

Maintain only 10 candidates.

Signal 2 — Continually changing best candidate

Examples:

```
always process smallest distance
always choose highest priority
always choose earliest finishing task
always take cheapest available resource
```

A heap lets us repeatedly find that candidate efficiently.

Signal 3 — Streaming data

Suppose values arrive continuously:

```
5
12
3
17
8
21
...
```

and the system asks:

> What are the largest 3 values seen so far?

We don't want to repeatedly sort all previous data.

Maintain:

```
min-heap of size 3
```

Memory becomes:

```
O(k)
```

instead of storing/sorting everything.

Signal 4 — Scheduling

Words such as:

```
next available
earliest deadline
minimum finishing time
highest-priority task
next meeting ending
available worker
```

are strong heap signals.

Signal 5 — Repeated minimum/maximum queries

If an algorithm repeatedly does:

```
Find minimum
remove minimum
insert something
find minimum
remove minimum
...
```

a heap is often better than repeatedly sorting.

---

8. Core Pattern 1 — Top K Largest

Suppose:

```
numbers = [4, 10, 2, 8, 15, 3]
k = 3
```

We want:

```
15, 10, 8
```

Maintain a **min-heap of size K**.

Why a min-heap?

Because among our current Top-K values, we care most about the weakest candidate.

```
Top 3 candidates:

[8, 15, 10]

smallest candidate = 8
```

If a new value is:

```
12
```

then:

```
12 > 8
```

So `8` should leave.

New Top 3:

```
10, 12, 15
```

The min-heap exposes `8` in `O(1)`.

---

9. Top-K Invariant

This is worth memorizing.

During processing:

> The heap contains the best K elements among all elements processed so far.

And:

```
heap[0]
```

is:

> The worst element among those K best elements.

That sounds strange initially, but it is the foundation of many Top-K problems.

---

10. Top-K Template

```
import heapq

def top_k_largest(nums, k):
    heap = []

    for num in nums:
        heapq.heappush(heap, num)

        if len(heap) > k:
            heapq.heappop(heap)

    return heap
```

Example:

```
top_k_largest([4, 10, 2, 8, 15, 3], 3)
```

The heap contains the three largest values.

The internal heap order is not guaranteed to be descending.

---

11. Complexity of Top-K Heap

Suppose:

```
n = total elements
k = required results
```

Each element may cause one heap operation:

```
O(log k)
```

Therefore:

```
Time  = O(n log k)
Space = O(k)
```

Compare with sorting:

```
O(n log n)
```

When:

```
k << n
```

the heap approach can be substantially better.

---

12. Core Pattern 2 — Streaming Selection

Imagine transactions arrive continuously:

```
Transaction stream
      |
      v
+-------------+
|   Min Heap  |
|   size = K  |
+-------------+
      |
      v
Top-K suspicious scores
```

Suppose:

```
k = 3
```

Stream:

```
5, 2, 10, 4, 12, 8
```

Start:

```
[]
```

5:

```
[5]
```

2:

```
[2, 5]
```

10:

```
[2, 5, 10]
```

4 arrives.

Heap is already size 3.

Compare:

```
4 > heap[0]
4 > 2
```

Replace 2:

```
[4, 5, 10]
```

12:

```
12 > 4

[5, 12, 10]
```

8:

```
8 > 5

[8, 12, 10]
```

Final values are:

```
8, 10, 12
```

---

13. Better Streaming Implementation

Instead of:

```
heappush(...)
heappop(...)
```

we can sometimes use:

```
heapq.heapreplace(heap, value)
```

Example:

```
if num > heap[0]:
    heapq.heapreplace(heap, num)
```

This removes the minimum and inserts the new element efficiently.

---

14. Core Pattern 3 — Scheduling

Imagine three jobs:

```
Job A finishes at 10
Job B finishes at 4
Job C finishes at 7
```

If the system repeatedly needs:

> Which job becomes available next?

Use:

```
min-heap ordered by finishing time
```

Heap conceptually:

```
      4
     / \
   10   7
```

The next available job is immediately:

```
heap[0] = 4
```

Scheduling problems often store tuples:

```
(end_time, resource_id)
```

Example:

```
heapq.heappush(heap, (15, "worker_3"))
```

Python compares the first tuple field first.

---

15. Common Scheduling Pattern

Suppose workers become available at:

```
worker A → time 12
worker B → time 5
worker C → time 9
```

Heap:

```
[
    (5, "B"),
    (12, "A"),
    (9, "C")
]
```

Pop:

```
available_time, worker = heapq.heappop(heap)
```

You get worker B because it becomes available first.

After assigning more work:

```
new_available_time = 14

heapq.heappush(heap, (14, "B"))
```

This pattern appears in:

- job schedulers
- meeting room allocation
- CPU simulation
- worker pools
- server assignment
- event simulation

---

16. Important Tuple Pattern

Often we need:

```
(priority, item)
```

For example:

```
heapq.heappush(heap, (2, "task-A"))
heapq.heappush(heap, (1, "task-B"))
```

Pop returns:

```
(1, "task-B")
```

For more complex objects, use a tie-breaker:

```
(priority, sequence_number, task)
```

Example:

```
(5, 0, task_a)
(5, 1, task_b)
```

Why?

If priorities are equal, Python otherwise may try to compare the task objects themselves.

---

17. Choosing Min-Heap vs Max-Heap

This frequently causes confusion.

K largest values

Use:

```
MIN heap of size K
```

because we want quick access to the smallest current winner.

K smallest values

Use conceptually:

```
MAX heap of size K
```

because we want quick access to the largest current winner.

In Python that commonly means negating values.

---

18. Medium Problem — Top K Frequent Elements

Problem

Given an integer array `nums` and integer `k`, return the `k` most frequent elements.

Example:

```
nums = [1,1,1,2,2,3]
k = 2
```

Frequency:

```
1 → 3
2 → 2
3 → 1
```

Result:

```
[1, 2]
```

The result order does not matter unless specifically required.

---

19. Recognition Signals

The problem says:

```
K most frequent
```

Immediately recognize:

```
Top-K problem
       +
frequency counting
```

This suggests two structures:

```
Hash Map
   +
Heap
```

Pipeline:

```
nums
 |
 v
frequency map
 |
 v
(element, frequency)
 |
 v
min-heap size K
 |
 v
Top K frequent
```

20. Brute-Force Reasoning

First count frequencies.

For:

```
[1,1,1,2,2,3]
```

create:

```
{
    1: 3,
    2: 2,
    3: 1
}
```

Then convert into something like:

```
[(1,3), (2,2), (3,1)]
```

Sort by frequency descending:

```
[(1,3), (2,2), (3,1)]
```

Take first `k`.

Brute-Force Pseudocode

```
count frequency of every number

create list of (number, frequency)

sort list by frequency descending

return first K numbers
```

Brute-Force Complexity

Let:

```
n = number of input values
m = number of distinct values
```

Frequency counting:

```
O(n)
```

Sorting unique values:

```
O(m log m)
```

Total:

```
O(n + m log m)
```

Worst case:

```
m = n
```

giving:

```
O(n log n)
```

Space:

```
O(m)
```

---

21. Optimized Heap Reasoning

We do not care about fully ordering all `m` unique values.

We need only:

```
K winners
```

Therefore maintain:

```
min-heap of size K
```

Each heap entry:

```
(frequency, number)
```

Why frequency first?

Because we want the heap ordered according to frequency.

Example:

```
(3, 1)
```

means:

```
number 1 occurs 3 times
```

---

22. Heap Invariant

After processing any number of unique elements:

> The heap contains the K highest-frequency elements encountered so far.

And:

```
heap[0]
```

represents the least frequent element among the current winners.

Therefore, once the heap exceeds size `k`:

```
heapq.heappop(heap)
```

removes the weakest candidate.

---

23. Optimized Pseudocode

```
frequency = count every element

heap = empty min-heap

for each number and count:
    push (count, number)

    if heap size > k:
        pop minimum-frequency element

result = numbers remaining in heap

return result
```

---

24. Dry Run

Input:

```
nums = [1,1,1,2,2,3]
k = 2
```

Frequency:

```
1 → 3
2 → 2
3 → 1
```

Start:

```
heap = []
```

Process 1:

```
push (3,1)

heap:
[(3,1)]
```

Process 2:

```
push (2,2)

heap:
[(2,2), (3,1)]
```

Heap size is 2.

Fine.

Process 3:

```
push (1,3)
```

Conceptually:

```
[(1,3), (3,1), (2,2)]
```

Size:

```
3 > k
```

Remove smallest frequency:

```
(1,3)
```

Remaining:

```
[(2,2), (3,1)]
```

Therefore result:

```
[2, 1]
```

which correctly represents the two most frequent elements.

---

25. Edge Cases

Edge Case 1 — One element

```
nums = [5]
k = 1
```

Result:

```
[5]
```

Edge Case 2 — Every value has same frequency

```
nums = [1,2,3]
k = 2
```

Any valid two values can be returned if the problem permits arbitrary order for ties.

Edge Case 3 — K equals unique count

```
nums = [1,1,2,3]
k = 3
```

Return every unique number.

Edge Case 4 — Negative numbers

```
nums = [-1,-1,-2,-3]
```

No special treatment is needed.

Dictionary keys and heap tuples handle negative integers normally.

Edge Case 5 — Very large input

This is where bounded-heap logic matters.

Instead of sorting potentially millions of unique values, the heap remains bounded by:

```
k
```

apart from the required frequency map.

---

26. Complexity of Optimized Heap Solution

Frequency map:

```
O(n)
```

For `m` unique values, each heap operation costs:

```
O(log k)
```

Therefore:

```
Time:
O(n + m log k)

Space:
O(m + k)
```

The map dominates memory:

```
O(m)
```

When:

```
k << m
```

the heap avoids sorting all unique values.

---

27. Python Solution

```
from collections import Counter
import heapq


def top_k_frequent(nums: list[int], k: int) -> list[int]:
    frequencies = Counter(nums)

    heap: list[tuple[int, int]] = []

    for number, frequency in frequencies.items():
        heapq.heappush(heap, (frequency, number))

        if len(heap) > k:
            heapq.heappop(heap)

    return [number for frequency, number in heap]
```

Example:

```
nums = [1, 1, 1, 2, 2, 3]
```

```
print(top_k_frequent(nums, 2))
```

Possible output:

```
[2, 1]
```

Both:

```
[1, 2]
```

and:

```
[2, 1]
```

are correct.

---

28. Why the Algorithm Is Correct

The correctness argument comes from the bounded heap invariant.

Suppose the heap currently contains the `k` highest-frequency elements encountered so far.

Now consider a new element with frequency `f`.

We insert it.

The heap temporarily contains:

```
k + 1
```

candidates.

Exactly one of these candidates cannot belong to the best K.

Which one?

The element with the smallest frequency.

The min-heap exposes exactly that element:

```
heapq.heappop(heap)
```

After removing it, the heap again contains the best `k` candidates.

Therefore, after every unique number has been processed, the heap contains exactly the `k` most frequent elements.

---

29. Why Not Use a Max Heap?

You might think:

> We need the biggest frequencies, so use a max heap.

You could.

But then you would normally store **all unique elements** and pop K times:

```
Heap size = m
```

That gives approximately:

```
O(m + k log m)
```

The bounded min-heap approach uses:

```
Heap size = k
```

and naturally discards bad candidates early.

The deeper Top-K principle is:

```
Want K largest
      ↓
keep K winners
      ↓
need fast access to weakest winner
      ↓
MIN HEAP
```

---

30. Alternative: Bucket Sort

There is an even better theoretical solution for this particular problem.

Frequency cannot exceed:

```
n
```

so we can create buckets:

```
frequency 1 → [...]
frequency 2 → [...]
frequency 3 → [...]
...
```

Then scan backward.

This can achieve:

```
O(n)
```

time.

But for **Day 13**, the heap solution is more useful because it teaches a reusable pattern applicable to:

- Top K prices
- Top K scores
- Top K nearest objects
- Top K frequent events
- streaming ranking
- bounded candidate selection

---

31. Common Mistake — Sorting the Heap

Suppose:

```
heap = [3, 5, 4, 10, 8]
```

Do not assume:

```
3 < 5 < 4 < 10 < 8
```

A heap is not sorted.

Only this is guaranteed:

```
heap[0] == minimum
```

If the final answer needs sorted output, sorting the K remaining values costs:

```
O(k log k)
```

which can still be much cheaper than sorting all `n` elements.

---

32. Common Mistake — Wrong Heap Direction

For:

```
K largest
```

students often create a max-heap of size K.

Ask instead:

> Which element must I efficiently remove when a better candidate arrives?

For K largest:

```
remove smallest winner
```

Therefore:

```
min-heap
```

For K smallest:

```
remove largest winner
```

Therefore:

```
max-heap
```

---

33. Common Mistake — Keeping More Than K Items

This:

```
for num in nums:
    heapq.heappush(heap, num)
```

creates an `O(n)` heap.

For Top-K streaming selection, the key invariant should be:

```
len(heap) <= k
```

after each iteration.

---

34. Common Mistake — Using `heapify()` Incorrectly

This works:

```
heap = [5, 3, 8, 1]

heapq.heapify(heap)
```

Afterward the list is a valid heap.

But don't expect:

```
heap == [1, 3, 5, 8]
```

Heapify creates a valid heap, not a fully sorted list.

---

35. Common Mistake — Tuple Comparisons

Consider:

```
heapq.heappush(heap, (5, custom_object_a))
heapq.heappush(heap, (5, custom_object_b))
```

If both priorities are `5`, Python might need to compare:

```
custom_object_a
vs
custom_object_b
```

which may fail.

Use:

```
(priority, counter, object)
```

Example:

```
import itertools

counter = itertools.count()

heapq.heappush(heap, (5, next(counter), custom_object_a))
heapq.heappush(heap, (5, next(counter), custom_object_b))
```

This is particularly useful in real job schedulers.

---

36. Scheduling Pattern to Remember

A very common heap pattern is:

```
sort events by start time
        |
        v
min-heap of active end times
        |
        +--> earliest ending resource
        |
        v
reuse or allocate resource
```

For example, Meeting Rooms II conceptually does:

```
Meetings:
[0,30]
[5,10]
[15,20]
```

At each meeting:

```
Which room becomes available first?
```

That's a heap question.

The heap stores:

```
meeting end times
```

not the full scheduling history.

---

37. Streaming Top-K Template to Memorize

```
import heapq


def top_k_stream(stream, k):
    heap = []

    for value in stream:
        if len(heap) < k:
            heapq.heappush(heap, value)

        elif value > heap[0]:
            heapq.heapreplace(heap, value)

    return heap
```

Its central invariant:

```
heap contains largest K values seen so far
```

with:

```
Time per new value = O(log k)
Memory             = O(k)
```

That makes it suitable for large or unbounded streams.

---

38. Pattern Comparison

Requirement	Best mental model	Typical heap
K largest	Keep winners, remove weakest	Min-heap size K
K smallest	Keep winners, remove weakest	Max-heap size K
Stream Top-K	Bounded candidate set	Heap size K
Earliest event	Next event first	Min-heap

Requirement	Best mental model	Typical heap
Highest priority	Best priority first	Min/max heap
Meeting rooms	Earliest finishing room	Min-heap
Merge sorted streams	Smallest current head	Min-heap
Dijkstra	Smallest known distance	Min-heap

---

39. Heap Complexity Cheat Sheet

For heap size `h`:

```
Peek:
O(1)

Push:
O(log h)

Pop:
O(log h)

Push + pop:
O(log h)

Heapify:
O(h)
```

For a Top-K problem:

```
h <= k
```

so operations become:

```
O(log k)
```

rather than:

```
O(log n)
```

That distinction is one of the main reasons bounded heaps are powerful.

---

40. Today's Mental Model

When you encounter a problem, run this decision process:

```
Do I repeatedly need smallest/largest?
              |
          yes
```

```
              |
              v
           Heap?


Does the problem say Top-K?
              |
           yes
              |
              v
Can I keep only K candidates?
              |
           yes
              |
              v
Bounded heap of size K


Do events/tasks compete by
priority/deadline/finish time?
              |
           yes
              |
              v
Priority Queue / Heap


Does data arrive continuously?
              |
           yes
              |
              v
Maintain heap incrementally
instead of repeatedly sorting
```

The key Day 13 takeaway is:

> **Sorting answers "put everything in order." A heap answers "keep giving me the next best item."**

And for Top-K:

> **Do not rank everything when you only need K winners. Maintain K candidates and efficiently remove the weakest one.**

Day 13 POC Story: Explainable, Fairness-Aware Abnormal Expense Review

Executive summary

This project is an interview-ready classical machine-learning system that prioritizes employee expense claims for human review. It predicts the probability that a later completed

audit will confirm an abnormal expense, then applies a business-cost and review-capacity policy to decide whether the claim should be reviewed, treated as uncertain, or recommended for clearing.

The project demonstrates more than model training. It includes a target definition, data contract, deterministic structured dataset, lineage, quality gates, leakage-safe time splitting, baseline-first experimentation, two tree models, an unsupervised anomaly model, tuning, probability calibration, cost-sensitive threshold selection, subgroup evaluation, uncertainty and abstention, three forms of explanation, a versioned FastAPI service, tests, security controls, observability, model/data cards, latency evidence, and timed demo scripts.

The selected model is balanced logistic regression. It was selected because it produced the best validation average precision after all default models were compared and the winning family was tuned. On the newest untouched test period, it achieved:

- ROC AUC: **0.8740**
- Average precision: **0.3785**, versus 6.67% test prevalence
- 95% bootstrap interval for average precision: **0.2997–0.4716**
- Review rate: **10.94%**, below the 12% capacity
- Review yield/precision: **38.58%**
- Abnormal-claim recall: **63.33%**
- Calibrated Brier score: **0.0484**
- Expected calibration error: **0.0134**
- Local in-process API P95 latency: **29.92 ms**
- Verification: **10 tests passed**

All performance and business numbers in this document come from generated project artifacts. The data is synthetic, so these results demonstrate engineering quality and workflow state—not production effectiveness or realized financial savings.

---

1. Problem statement

Finance teams receive more employee expense claims than they can manually inspect. Reviewing every claim is expensive and slow, but reviewing too few can miss duplicate, unsupported, policy-breaking, or otherwise abnormal expenses.

The ML problem is:

> Given information available when an expense is submitted, estimate the probability that a later completed audit will confirm the claim as abnormal, and use that probability to prioritize a capacity-constrained human-review queue.

Target definition

The binary target is `is_abnormal`:

- `1`: a post-submission audit confirms a material policy violation, duplicate, fabricated evidence, or another abnormal condition.
- `0`: the completed audit does not confirm an abnormal condition.

This definition matters because “high model risk” is not the same as fraud. The system only predicts a later audit outcome and never establishes employee intent or guilt.

Users

- **Finance reviewers:** receive a prioritized review queue and reason codes.
- **Compliance analysts:** inspect policy performance, fairness slices, auditability, and limitations.
- **ML engineers:** train, evaluate, calibrate, version, and monitor the system.
- **Platform engineers:** operate the API, authentication, telemetry, deployment, and rollback.

Business value

The system concentrates reviewers on claims with higher predicted abnormality while keeping workload below a declared capacity. It supports:

- Higher abnormal-claim capture per reviewer hour.
- More consistent review prioritization.
- Explicit cost and capacity tradeoffs.
- Traceable model, data, schema, and policy versions.
- Human-readable evidence for each recommendation.

Scope

- Structured expense data available at submission time.
- Offline chronological training and evaluation.
- Probability calibration and cost-sensitive thresholding.
- Human-review recommendation through a synchronous API.
- Descriptive fairness checks and model-behavior explanations.

Non-goals

- Automatic reimbursement rejection.
- Fraud adjudication, accusation, or disciplinary action.
- Receipt-image or identity verification.
- Employment decisions.
- Causal conclusions about why an expense is abnormal.
- A production-readiness claim based on synthetic data.

---

2. Why abnormal-expense detection was chosen

The preparation brief allowed duplicate/abnormal expenses, invoice approval risk, payment-delay risk, or budget-overrun classification. Abnormal expense review was selected because it naturally supports every required ML-system capability:

1. It is a supervised classification problem when audit labels exist.
2. It supports an unsupervised anomaly benchmark when labels are limited.
3. Errors have asymmetric business costs: missing an abnormal claim is more expensive than reviewing a normal one.

4. Finance review capacity creates a concrete threshold-selection constraint.
5. Individual recommendations require local explanations and audit fields.
6. Employee-related data makes subgroup analysis and governance essential.
7. Human review provides a safe abstention path.

This creates a stronger interview story than treating the task as a standalone Kaggle-style accuracy exercise.

---

3. Solution in one sentence

A deterministic, leakage-controlled training pipeline compares classical supervised and unsupervised models, calibrates the selected probability model, chooses a review cutoff from explicit costs and capacity, audits subgroup behavior, generates explanations, and serves versioned human-review recommendations through FastAPI.

---

4. Requirements

Functional requirements

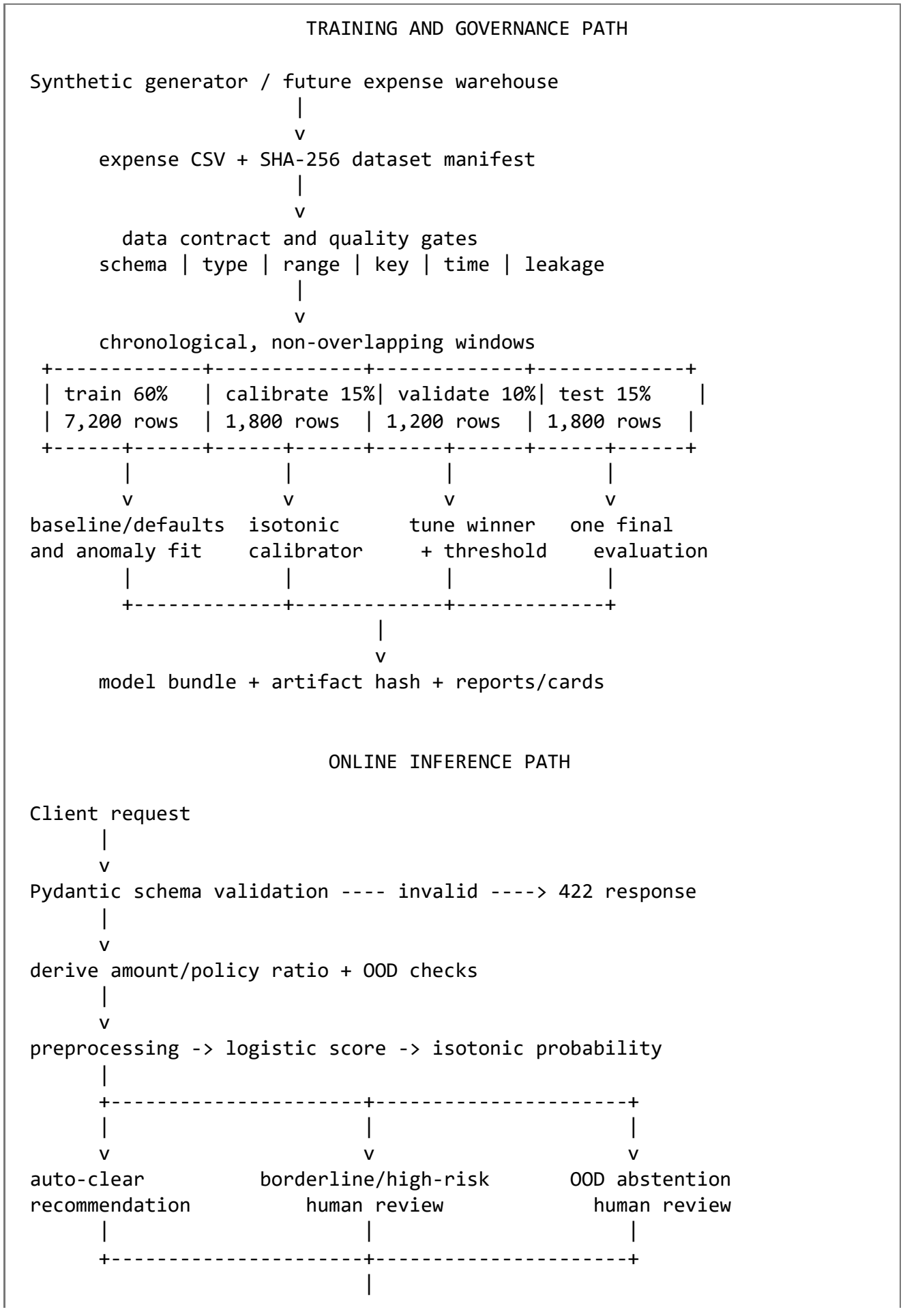
1. Generate or ingest realistic structured expense data.
2. Enforce a versioned data contract and quality rules.
3. Record dataset lineage, version, time range, target rate, and hash.
4. Prevent target, post-outcome, ID, and protected-field leakage.
5. Split data chronologically into train, calibration, validation, and test.
6. Establish a dummy and logistic baseline before tuning.
7. Compare logistic regression, random forest, histogram gradient boosting, and Isolation Forest.
8. Log every baseline, default candidate, and tuning experiment.
9. Calibrate the selected model on an independent calibration window.
10. Select a threshold using false-negative cost, false-positive review cost, and reviewer capacity.
11. Measure ranking, probability, operational, subgroup, uncertainty, and latency outcomes.
12. Produce global, local, and counterfactual explanations.
13. Serve predictions with reason codes and audit metadata.
14. Test data, artifact, model-policy, security, abstention, and API behavior.

Non-functional requirements

- Reproducible CPU-only execution with a fixed seed.
- Immutable hashes for the dataset and model artifact.
- Strict request validation and deterministic derived features.
- P95 local API latency target below 100 ms after warm-up.
- No raw input fields in application logs.
- No gender in training or inference.
- Generic server failures without internal exception leakage.
- Human review for high-risk, borderline, and sufficiently OOD cases.

---

5. End-to-end architecture



```
                                  v
reason codes + warnings + versions + UUID + UTC time + feature hash
```

Component ownership

Component	Responsibility
Generator	Creates deterministic realistic structured claims without real personal data
Data contract	Defines exact fields, types, ranges, allowed values, target, and exclusions
Quality gates	Reject malformed, incomplete, duplicated, temporally invalid, or leakage-prone data
Experiment pipeline	Establishes baselines, compares candidates, tunes winner, and logs every run
Calibrator	Converts raw model scores into operationally meaningful probabilities
Policy layer	Converts probability into review, abstention, or clear recommendation
Explanation layer	Produces global, local, and counterfactual model-behavior evidence
FastAPI	Validates requests and returns predictions plus audit state
Reports/cards	Preserve model, data, fairness, calibration, and evaluation evidence

---

6. Thought process and key design decisions

6.1 Human decision support instead of automated rejection

Expense risk is a high-impact employee context. A false positive can inconvenience or unfairly implicate someone. Therefore, the model only recommends review. High risk, borderline uncertainty, and distribution warnings all terminate in a human decision.

6.2 A precise post-audit target instead of a vague “fraud” label

“Fraud” is legally and operationally ambiguous. The target is defined as an audit-confirmed abnormal expense. This produces a label that a finance organization could operationalize and avoids claiming the model detects intent.

6.3 Synthetic data with explicit limitations

No production expense warehouse or audit system was available. The core capability therefore uses a deterministic synthetic generator rather than mocks or fabricated report numbers. The generator creates plausible interactions—policy exceedance, missing receipt, recent duplicates, risky merchants, cross-border activity, late submission, and claim velocity—while retaining stochastic label noise.

Synthetic performance proves that the repository works end to end. It does not prove that the model will perform on real employee claims.

6.4 Chronological splitting instead of random splitting

A random split can let near-contemporaneous behavioral patterns appear in both training and test data, producing optimistic results. The production question is “Can a model trained on history rank future claims?” The project therefore uses four chronological windows:

Split	Rows	Period	Target rate	Purpose
Train	7,200	2024-01-01 to 2025-03-15	6.50%	Fit models
Calibration	1,800	2025-03-15 to 2025-07-03	6.44%	Fit isotonic calibrator
Validation	1,200	2025-07-03 to 2025-09-15	7.08%	Model selection, tuning, threshold
Test	1,800	2025-09-15 to 2025-12-30	6.67%	Final untouched evaluation

The four-way split prevents the same validation window from simultaneously training the model, fitting calibration, selecting a threshold, and claiming final performance.

6.5 Baseline before tuning

The experiment order is deliberate:

1. Prior-probability dummy baseline.
2. Untuned logistic regression.
3. Untuned random forest.
4. Untuned histogram gradient boosting.
5. Label-free Isolation Forest.
6. Hyperparameter candidates for only the best default supervised family.

This prevents premature optimization and makes the incremental value of modeling measurable.

6.6 Choose the simplest winner, not the most complex model

Both tree models were competitive, but untuned logistic regression had the strongest validation average precision. Balanced logistic regression then improved it further. Selecting it provides:

- Strongest measured validation ranking.
- Lower inference cost.
- Easier operational explanation.
- More stable behavior on a medium-sized structured dataset.
- A useful interview lesson: complexity is not a selection criterion; evidence is.

6.7 Average precision as the main selection metric

Only about 6–7% of claims are abnormal. Accuracy would be misleading because predicting every claim as normal would already be highly accurate. Average precision focuses on positive-class ranking and precision-recall tradeoffs in an imbalanced setting. ROC AUC remains useful but is not sufficient by itself.

6.8 Calibrate after selection

The raw balanced-logistic score ranks well but is not a reliable operational probability. Isotonic regression is fitted only on the calibration split. Calibration dramatically improves Brier score and expected calibration error, which makes cost-based thresholding more defensible.

The tradeoff is that isotonic regression creates stepwise probabilities and ties. This slightly lowers average precision and can make a one-feature calibrated probability delta equal zero even when the underlying raw model score moves. The API therefore exposes both raw-score and calibrated-probability explanation deltas.

6.9 Business threshold instead of 0.5

A default threshold of 0.5 ignores class imbalance, reviewer capacity, and error cost. The project scans validation probabilities and minimizes:

```
total policy cost = 500 USD × false negatives
                  + 15 USD × reviewed normal claims
```

The cutoff must also satisfy a buffered capacity. Maximum review capacity is 12%, while threshold selection uses a one-percentage-point buffer to reduce the chance of exceeding capacity under modest time drift.

6.10 Abstention as a first-class outcome

The model can decline to make a normal automated recommendation:

- Probabilities between the review cutoff and cutoff + 0.04 are marked borderline and sent to human review.
- At least two input distribution warnings trigger OOD abstention regardless of score.
- High-risk in-distribution scores go to standard high-risk review.
- Only sufficiently low-risk, in-distribution claims receive an auto-clear recommendation.

This is safer than pretending every probability is equally reliable.

6.11 Model-agnostic explanations

The explanation framework works even if the selected model family changes:

- Permutation importance for global behavior.
- Reference perturbation for a single claim.
- Constrained scenario search for counterfactuals.

SHAP was not required for the core capability. The chosen approach keeps dependencies smaller and makes the mechanics easy to explain, while documenting that perturbation methods are not causal and can be distorted by correlated inputs.

---

7. Data design, contract, lineage, and quality

Dataset state

Field	Value
Dataset version	`expenses-synthetic-v1`
Schema version	`1.0.0`
Rows	12,000
Columns	25
Event range	2024-01-01 through 2025-12-30
Overall target rate	6.575%
Generator seed	45
Dataset SHA-256	`2e41e50a9cec73f84e1ee5d224826bd245ae7fb579b0bcf3205e0a81c5bceb42`

Feature groups

Group	Important fields
Claim amount and policy	`amount_usd`, `policy_limit_usd`, derived `amount_to_policy_ratio`
Submission timing	`days_since_expense`, weekend, outside-business-hours
Employee history	prior 30-day claim count and total, tenure
Evidence and duplication	receipt attached, duplicate count in seven days
Operational context	cross-border, region, department, employee level
Claim categorization	expense category, merchant risk, country risk, payment method

Excluded fields

- `is_abnormal`: target leakage.
- `audit_completed_at`: post-outcome timestamp.
- `employee_id` and `expense_id`: identifiers rather than generalizable behavior.
- `employee_gender`: evaluation-only protected field.
- `submitted_at`: used for temporal splitting, not direct prediction.

Why derive amount-to-policy ratio

An amount of $500 has different meaning under a $100 meal limit and a $1,400 airfare limit. The ratio expresses policy exceedance consistently. It is stored in training data and deterministically derived by the API so callers cannot provide an inconsistent amount/ratio pair.

Data-quality results

All configured gates passed:

- 12,000 rows found.

- Zero missing required fields.
- Zero duplicated primary keys.
- No missing or unexpected columns.
- Event timestamps are monotonically ordered.
- Every audit completion is after its submission.
- Target rate is within the allowed range.
- No excluded columns appear in model features.
- Dataset bytes match the manifest SHA-256.

Lineage

```
project_config seed
        |
        v
deterministic generator
        |
        v
chronologically sorted CSV
        |
        +--> SHA-256/version/row/time/prevalence manifest
        |
        v
contract checks -> split manifest -> experiments -> model/report artifacts
```

No real personal data or external service was used.

---

8. Preprocessing and modeling

Preprocessing

- Numeric and Boolean fields: median imputation followed by standard scaling.
- Categorical fields: most-frequent imputation followed by one-hot encoding.
- Unknown categorical values: ignored by the encoder, while the inference layer separately produces an OOD warning.
- Transformations are inside the fitted scikit-learn pipeline so training and serving use identical logic.

Even though generated data currently has no missing values, imputation makes the pipeline structurally robust and documents expected behavior for future sources.

Model comparison on validation

Model	Stage	ROC AUC	Average precision
Prior dummy	Baseline	0.5000	0.0708
Logistic regression	Untuned candidate	0.8396	0.3840
Random forest	Untuned candidate	0.8315	0.3738
Histogram gradient boosting	Untuned candidate	0.8368	0.3649

Model	Stage	ROC AUC	Average precision
Isolation Forest	Unsupervised candidate	0.6744	0.1589
Logistic, `C=0.3`, unbalanced	Tuning candidate	0.8399	0.3835
Logistic, `C=1`, unbalanced	Tuning candidate	0.8396	0.3840
Logistic, `C=1`, balanced	**Selected**	**0.8439**	**0.3961**
Logistic, `C=3`, unbalanced	Tuning candidate	0.8389	0.3830

Nine experiments were recorded. Isolation Forest did not use labels during fitting; labels were used only afterward to evaluate how well its anomaly score ranked audit-confirmed abnormal claims.

Selected model artifact

Field	Value
Model version	`expense-risk-v1`
Family	Logistic regression
Parameters	`C=1.0`, `class_weight=balanced`
Dataset version	`expenses-synthetic-v1`
Schema version	`1.0.0`
Artifact SHA-256	`017b44b438d06c5db9111cff9135b210c9ec3ec3b048573869634a49e62cb88f`

The serialized bundle contains preprocessing, estimator, calibrator, feature order, training reference values, numeric ranges, known categories, threshold policy, explanation caveat, and all versions.

---

9. Detailed evaluation report

Test-set classification and ranking

The test set contains 1,800 claims, including 120 abnormal claims.

Metric	Result
ROC AUC	0.8740
Calibrated average precision	0.3785
Raw average precision	0.4245
AP bootstrap 95% interval	0.2997–0.4716
Log loss	0.1740
F1 at operational cutoff	0.4795

The average precision is substantially higher than the 6.67% positive prevalence, showing that the model ranks abnormal claims above random selection.

Test confusion matrix at the operational cutoff

```
                        Predicted clear   Predicted review
Actually normal             1,559              121
Actually abnormal              44               76
```

From these measured counts:

```
reviewed claims = 121 + 76 = 197
review rate     = 197 / 1,800 = 10.94%
review yield    = 76 / 197 = 38.58%
abnormal recall = 76 / 120 = 63.33%
```

Business-cost calculation

The declared scenario assumes:

- $500 for each missed abnormal claim.
- $15 reviewer cost for each normal claim sent to review.

Measured test calculation:

```
false-negative cost = 44 × $500 = $22,000
false-positive cost = 121 × $15 =  $1,815
total policy cost                 = $23,815

no-review cost      = 120 × $500 = $60,000
estimated avoided cost           = $60,000 - $23,815
                                 = $36,185
```

The expected policy cost per 1,000 claims is $13,230.56. These are scenario-model outputs, not booked savings. A real deployment must estimate costs from reviewer time, recovery rate, claim value, appeal cost, employee friction, and downstream impact.

Threshold state

Policy value	Value
Review threshold	0.174603
Maximum review rate	12.00%
Selection buffer	1 percentage point
Validation review rate	9.17%
Test review rate	10.94%
Borderline width	0.04 probability

The threshold was selected only on validation data. Test capacity was checked afterward, not used to choose the cutoff.

---

10. Calibration and uncertainty

Why calibration matters

A risk-ranking model can order claims correctly while producing unreliable numerical probabilities. Because the policy assigns business costs to probabilities and thresholds, probability quality matters in addition to ranking.

Measured calibration change

Metric	Raw model	Isotonic calibrated
Brier score	0.1414	0.0484
Expected calibration error	0.2574	0.0134
ROC AUC	0.8776	0.8740
Average precision	0.4245	0.3785

Lower Brier score and lower expected calibration error are better. Ranking metrics fall slightly because isotonic regression maps ranges of raw scores to the same stepwise probability.

Statistical uncertainty

Average precision uses 300 nonparametric bootstrap resamples. The measured 95% interval is 0.2997–0.4716. The interval communicates that performance is estimated from a finite test sample rather than known exactly.

Operational uncertainty

The service returns an explicit uncertainty state:

- `in_distribution`
- `borderline_probability`
- `out_of_distribution`

At least two learned-range/category warnings trigger `manual_review_ood_abstention`. Borderline scores trigger `manual_review_borderline_abstention`. Neither case is silently treated as low risk.

---

11. Explainability

Global explanation

Permutation importance measures the decrease in held-out average precision when one input is shuffled. The leading measured features were:

Feature	Mean AP decrease when permuted
Amount-to-policy ratio	0.2871
Receipt attached	0.1117
Duplicate count in seven days	0.0788

Feature	Mean AP decrease when permuted
Policy limit	0.0242
Merchant risk tier	0.0107
Cross-border indicator	0.0088
Outside-business-hours indicator	0.0077

Permutation importance is not causality. Correlated fields can share, hide, or transfer importance.

Local explanation

For one claim, the system replaces each feature with its training median or mode, scores all perturbations in a single vectorized batch, and reports the largest changes.

Each local reason contains:

- Feature and stable reason code.
- Observed value.
- Training reference value.
- Raw model-probability delta.
- Calibrated-probability delta.
- Whether the observed value raises or lowers modeled risk.

Raw and calibrated deltas are both shown because isotonic calibration may produce a zero calibrated delta across a flat step even when the underlying model changes.

Counterfactual explanation

The counterfactual search considers up to three constrained scenarios, such as:

- Attach valid receipt evidence.
- Resolve a genuinely erroneous duplicate link.
- Use a corporate card for a future comparable claim.
- Submit a future claim promptly.
- Use a verified lower-risk merchant in a future comparable situation.

The output says whether a scenario crosses below the review cutoff. A proposal that does not cross is reported honestly. These scenarios explain model behavior and are not instructions to rewrite truthful historical facts.

Explanation caveats

- Association is not causation.
- Reasons do not prove wrongdoing.
- One-feature perturbations may create unlikely combinations.
- Correlated features can distort attribution.
- Counterfactual feasibility requires finance and policy review.

---

12. Fairness evaluation

Protected-field strategy

`employee_gender` is included only in the synthetic offline evaluation data. It is excluded from preprocessing, model fitting, artifact features, and the inference request.

Gender slices

Group	Rows	Positives	Review rate	Review precision	Recall
Female	833	60	12.00%	40.00%	66.67%
Male	938	58	10.23%	36.46%	60.34%
Nonbinary	29	2	3.45%	100.00%	50.00%

The Nonbinary slice is explicitly low-support: 29 records and two positives cannot support a reliable conclusion. The overall gender recall max-minus-min is 16.67 percentage points, but that number is dominated by the low-support group. Even the better-supported group comparison should be treated as descriptive because the data is synthetic.

Region slices

Region	Rows	Positives	Review rate	Recall
APAC	569	50	13.88%	64.00%
EMEA	430	21	10.93%	61.90%
LATAM	275	21	10.18%	66.67%
North America	526	28	8.17%	60.71%

The measured regional recall gap is 5.95 percentage points. Region remains a modeled operational field, so its use needs a real legal, policy, proxy-risk, and necessity review before deployment.

What this fairness analysis can and cannot say

It can identify differences in selection rate, recall, false-positive rate, prevalence, and support. It cannot establish legal fairness, absence of discrimination, causal fairness, or individual fairness—especially with synthetic and low-support groups.

---

13. FastAPI serving design

Endpoints

Endpoint	Purpose
`GET /health`	Readiness and model/data/schema versions
`POST /v1/predict`	Validate and score one expense claim
`GET /metrics`	Process-local request, error, and P95 latency state
`/docs`	Generated OpenAPI interface

Input behavior

- Extra fields are forbidden.
- Numeric ranges and categorical values are validated.
- `employee_gender` is rejected because it is not in the request contract.
- `amount_to_policy_ratio` is computed from amount and policy limit.
- The caller cannot supply target, employee ID, expense ID, or audit timestamp.

Output and audit fields

Every successful prediction includes:

- Calibrated abnormal probability.
- Operational review threshold.
- Decision and uncertainty status.
- Up to three reason codes and explanations.
- Distribution warnings.
- Request UUID and UTC scoring time.
- Model, dataset, schema, and policy versions.
- SHA-256 of the canonical input payload.

Decision states

```
manual_review_ood_abstention
review_high_risk
manual_review_borderline_abstention
auto_clear_recommendation
```

“Auto-clear” remains a recommendation rather than an irreversible reimbursement decision.

Error handling

- `422`: invalid range, category, missing field, or forbidden extra field.
- `401`: invalid or missing key when `EXPENSE_API_KEY` is configured.
- `503`: model artifact missing or unreadable.
- `500`: unexpected scoring error, returned without internal stack details.

Security

Implemented POC controls:

- Optional constant-time API-key comparison.
- Strict input schema.
- No protected field in the request.
- No raw feature values in structured prediction logs.
- Input hash for traceability.
- Generic error responses at the scoring boundary.

Production additions still required:

- TLS, gateway authentication, and role authorization.

- Secret manager and key rotation.
- Rate limits and abuse protection.
- Durable, access-controlled audit storage and retention.
- Dependency/container scanning and threat modeling.

Observability

The POC records request count, error count, recent latency, decision, request ID, version, and feature hash. A production service should also monitor:

- Feature and score drift.
- Calibration drift and delayed-label performance.
- Review rate and queue capacity.
- Review yield and abnormal recall.
- Subgroup recall and false-positive rate.
- Reviewer overrides and disagreement.
- OOD and abstention rates.
- Availability, saturation, and error budgets.

---

14. Latency and operational evidence

The latest local benchmark used FastAPI `TestClient`, 10 warm-up calls, and 150 measured sequential requests.

Metric	Result
Failures	0 of 150
Mean	26.97 ms
P50	26.29 ms
P95	29.92 ms
P99	36.43 ms
Maximum	67.20 ms
Sequential throughput	37.07 requests/second
P95 target	Below 100 ms — passed

This is an in-process Windows benchmark. It excludes network, API gateway, concurrent load, serialization across services, production logging, autoscaling, and cold starts. It proves local implementation efficiency, not production capacity.

---

15. Testing and verification

Ten tests pass. They cover:

1. Dataset manifest hash and row count.
2. Full data-contract success.
3. Chronological split separation and row preservation.

4. Protected, identifier, target, and post-outcome feature exclusions.
5. Prediction probability range and required audit/version fields.
6. Multiple distribution warnings causing OOD abstention.
7. Health endpoint and successful prediction.
8. Rejection of invalid amount and protected extra field.
9. API-key enforcement when configured.
10. Test capacity, quality target, artifact hash, and model version.

The final integrity check also recompiles source, scripts, and tests; verifies dataset and artifact hashes; and asserts the declared quality and latency targets passed.

---

16. Repository structure

```
Week-2/
|-- README.md
|-- Day-13-poc-story.md
|-- requirements.txt
|-- pyproject.toml
|-- configs/
|   |-- data_contract.json
|   `-- project_config.json
|-- data/
|   |-- raw/expenses_v1.csv
|   `-- manifests/expenses_v1.manifest.json
|-- src/expense_ml/
|   |-- data.py                 # deterministic generator and manifest
|   |-- quality.py              # data-contract gates
|   |-- modeling.py             # preprocessing, models, metrics,
experiment log
|   |-- evaluation.py           # thresholds, calibration, fairness,
uncertainty
|   |-- explain.py              # global/local/counterfactual explanations
|   |-- inference.py            # OOD, policy, audit response
|   |-- api.py                  # FastAPI endpoints and security boundary
|   `-- train.py                # full training/evaluation orchestration
|-- experiments/
|   `-- experiments.jsonl
|-- artifacts/model/
|   |-- expense_risk_v1.joblib
|   `-- artifact_manifest.json
|-- reports/
|   |-- evaluation.json / evaluation_report.md
|   |-- calibration_plot.png / calibration tables
|   |-- fairness_slices.csv / fairness_disparities.json
|   |-- global_permutation_importance.csv
|   |-- local_explanations.json
|   |-- counterfactual_explanations.json
|   |-- data_card.md / model_card.md
|   |-- data_quality_report.json / split_manifest.json
```

```
|   `-- latency_report.json
|-- scripts/
|   |-- run_all.ps1
|   |-- generate_data.py / train.py
|   |-- benchmark_api.py / refresh_explanations.py
|   `-- demo.py
|-- tests/
`-- docs/
```

---

17. How to run the complete project

From the project root:

```
cd E:\print-out\AI-45\POC\Week-2
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\scripts\run_all.ps1
```

The full workflow:

1. Regenerates deterministic data and its manifest.
2. Validates the dataset.
3. Trains baselines and candidates.
4. Tunes the selected family.
5. Calibrates and chooses the threshold.
6. Generates evaluation, fairness, calibration, and explanation artifacts.
7. Serializes and hashes the model.
8. Benchmarks the API.
9. Runs the tests.

Run the live prepared demo:

```
$env:PYTHONPATH = "src"
.\.venv\Scripts\python.exe scripts\demo.py
```

Run the API:

```
$env:PYTHONPATH = "src"
$env:EXPENSE_API_KEY = "replace-in-production"
.\.venv\Scripts\python.exe -m uvicorn expense_ml.api:app --host 127.0.0.1 --port 8000
```

---

18. Three-to-five-minute business story

Opening

“Finance cannot manually inspect every employee expense. I built a human-in-the-loop system that prioritizes claims likely to be confirmed abnormal by a later audit. It never

determines fraud or rejects a claim automatically.”

Business rule

“The policy assumes a missed abnormal claim costs $500 and reviewing a normal claim costs $15, with reviewer capacity capped at 12%. I calibrate probabilities and select the validation threshold that minimizes this cost under capacity.”

Evidence

“On 1,800 newest claims, the system reviews 10.94%, captures 63.33% of abnormal claims, and produces a 38.58% review yield. Under the scenario costs, that is $36,185 lower than reviewing none. Those are synthetic scenario results, not realized savings.”

Live prediction

Run `scripts/demo.py`. Point out probability, review decision, recent-duplicate/receipt/policy reason codes, no OOD warnings, and the model/data/schema/policy versions plus request hash.

Trust and close

“Gender is excluded from prediction, slice performance is reported with low-support warnings, uncertain/OOD claims abstain to humans, and the service has reproducible hashes and tests. Production still requires real target validation, legal review, shadow deployment, drift monitoring, and validated costs.”

---

19. Five-to-ten-minute technical story

1. **Contract and target:** audit-confirmed abnormal label, exact JSON schema, post-outcome and protected exclusions.
2. **Lineage:** deterministic seed, CSV hash, schema/data versions, quality report.
3. **Leakage control:** 60/15/10/15 chronological split with calibration and threshold windows separated.
4. **Experiments:** dummy first; logistic, random forest, gradient boosting; label-free Isolation Forest; tune only the default winner.
5. **Selection:** balanced logistic wins validation AP at 0.3961; simpler model retained on evidence.
6. **Calibration:** raw-to-isotonic Brier 0.1414 to 0.0484 and ECE 0.2574 to 0.0134; explain ranking-tie tradeoff.
7. **Policy:** explicit $500/$15 costs and 12% capacity; show confusion-matrix arithmetic.
8. **Uncertainty:** 300-sample AP bootstrap, borderline abstention, multi-warning OOD abstention.
9. **Explainability:** permutation, local raw/calibrated perturbation, constrained counterfactual; all non-causal.
10. **Fairness:** gender excluded, region audited, rare-group support caveat.
11. **Serving:** strict FastAPI contract, derived ratio, optional API key, versions, feature hash, no raw logging.
12. **Evidence:** 10 tests, artifact hashes, 29.92 ms local P95, and explicit production benchmark limitations.

20. Limitations and risks

1. **Synthetic-data realism:** generated relationships are simpler and cleaner than real finance behavior.
2. **Label quality:** there is no investigator disagreement, appeal, label censoring, or audit-selection bias.
3. **Feedback loops:** prioritized review changes which claims receive labels and can bias future training.
4. **Calibration steps:** isotonic mapping creates ties and flat local calibrated deltas.
5. **Fixed threshold:** prevalence or score drift can push review workload away from the intended capacity.
6. **Fairness evidence:** synthetic and small subgroups cannot establish real-world equity.
7. **Feature proxies:** region, department, and level may correlate with protected or organizational attributes.
8. **Counterfactual realism:** independent perturbations can violate feature relationships.
9. **Latency scope:** the benchmark does not include concurrent production traffic or network overhead.
10. **Security scope:** a demo API key is not production identity, authorization, or secrets management.
11. **Financial assumptions:** $500 and $15 are illustrative and require operational validation.
12. **Human factors:** reviewer consistency, override reasons, employee appeals, and reviewer fatigue are not modeled.

---

21. Production next steps

Data and labels

- Agree on target semantics and audit-completion windows with finance/compliance.
- Measure missingness, audit-selection bias, resubmissions, delayed labels, and investigator disagreement.
- Create point-in-time correct history features from a governed feature pipeline.
- Add repeated temporal backtests across business cycles.

Modeling and policy

- Validate cost assumptions and recovery amounts.
- Compare Platt, isotonic, and potentially segment-aware calibration over time.
- Add top-k batch queue enforcement alongside the score cutoff.
- Measure reviewer overrides, appeals, and decision consistency.
- Define retraining, promotion, rollback, and challenger gates.

Fairness and governance

- Obtain legal guidance on protected classes and permitted feature use.
- Add confidence intervals and minimum-support policies for subgroup decisions.
- Evaluate intersections such as region × level, subject to privacy and support.
- Establish documentation, review, and escalation processes for adverse outcomes.

Platform and monitoring

- Containerize and deploy behind an authenticated TLS gateway.
- Store audit events durably with access controls and retention policies.
- Add concurrent load, soak, failure, and cold-start tests.
- Monitor feature/score drift, calibration, yield, capacity, overrides, OOD, and subgroup performance.
- Shadow deploy before allowing any operational routing.

---

22. Likely interviewer questions and concise answers

Why not use accuracy?

Only 6.67% of test claims are abnormal. A mostly-normal prediction can have high accuracy without helping reviewers. Average precision and review yield better evaluate rare-positive ranking.

Why chronological instead of random splitting?

The real task predicts future claims from historical data. Chronological windows better expose time drift and prevent optimistic mixing of near-contemporaneous behavior.

Why four splits?

Train fits model parameters, calibration maps scores to probabilities, validation selects family/hyperparameters/threshold, and test provides one untouched final estimate. Combining these roles would leak selection decisions into reported performance.

Why did logistic regression beat the trees?

The engineered structured signals, especially policy ratio, are strongly separable with additive effects. Logistic regression had the highest measured validation AP. I chose evidence and operational simplicity rather than assuming trees must win.

Why include Isolation Forest?

It provides a label-free anomaly benchmark for cold-start or scarce-label conditions. Its AP of 0.1589 was better than the dummy but far below supervised candidates, so it was not deployed.

Why class weighting?

The positive class is rare. Balanced weighting improved validation average precision from 0.3840 to 0.3961 and therefore won the declared selection rule.

Why not use 0.5 as the threshold?

Probability 0.5 has no special connection to review capacity or asymmetric costs. The chosen 0.174603 cutoff minimizes validation scenario cost under the capacity constraint.

Why calibrate, and why did AP fall?

Cost decisions require meaningful probabilities. Isotonic calibration improved Brier and ECE substantially. It is stepwise and introduces ties, so ranking AP can fall even while probability reliability improves.

How do you prevent leakage?

The contract explicitly excludes target, audit completion time, IDs, and gender. All history fields are defined as prior to submission, splitting is chronological, and preprocessing is fitted within the training pipeline.

How is uncertainty handled?

Statistical performance gets a bootstrap interval. Operationally, borderline probabilities and multiple distribution warnings abstain to human review.

Are the explanations causal?

No. Permutation and reference perturbation describe model behavior. Correlation and unrealistic feature combinations can alter attribution. Counterfactuals are constrained scenarios, not proof or advice to modify records.

Is the model fair?

The project reports descriptive slice metrics and excludes gender from training, but synthetic data and a 29-row Nonbinary slice cannot establish fairness. Real deployment needs legal requirements, representative data, uncertainty intervals, proxy analysis, and ongoing audits.

What would you monitor?

Inputs, scores, calibration, delayed-label AP/recall, reviewer workload, yield, OOD/abstention, overrides, subgroup recall/FPR, latency, errors, and data/model/schema version consistency.

What happens if reviewer demand exceeds 12%?

The POC uses a buffered fixed cutoff. Production should additionally use a monitored top-k queue or daily capacity allocator, with escalation rules for OOD and critical policies.

How would you prove business value?

Validate costs, shadow-score live claims, compare review yield and recovery against current operations or a randomized controlled allocation, include employee/reviewer friction, and measure realized rather than assumed recovery.

---

23. What this project demonstrates for a Senior Applied AI/ML Engineer role

ML judgment

- Starts with target and decision design rather than an algorithm.
- Uses a metric appropriate for imbalance.
- Compares simple, tree-based, and unsupervised methods.
- Tunes only after establishing baselines.
- Selects a simpler model when it wins on evidence.

Production thinking

- Treats preprocessing and calibration as model artifacts.

- Versions schema, data, model, and policy separately.
- Uses hashes, audit fields, validation, health, telemetry, and tests.
- Measures quality, latency, workload, yield, recall, and scenario cost.

Responsible AI

- Keeps humans in the decision loop.
- Excludes gender from prediction.
- Reports subgroup support and disparities.
- Implements uncertainty and OOD abstention.
- Documents explanation and counterfactual caveats.

Communication

- Connects metrics to reviewer workflow and business costs.
- Separates measured results, assumptions, and limitations.
- Provides both a business demo and a technical deep dive.

---

24. Evidence map

Question	Source of truth
What is the project and how do I run it?	`README.md`
What was designed before implementation?	`docs/01_problem_and_design.md`
What is the exact schema and target?	`configs/data_contract.json`
What data was used?	`data/manifests/expenses_v1.manifest.json` and `reports/data_card.md`
Did data quality pass?	`reports/data_quality_report.json`
Are time splits leakage-safe?	`reports/split_manifest.json`
What experiments ran?	`experiments/experiments.jsonl`
Which artifact is deployed?	`artifacts/model/artifact_manifest.json`
What are final measured metrics?	`reports/evaluation.json` and `reports/evaluation_report.md`
Is it calibrated?	`reports/calibration_plot.png` and calibration CSV files
What are subgroup results?	`reports/fairness_slices.csv` and `reports/fairness_disparities.json`
What explains model behavior?	global importance CSV and local/counterfactual JSON reports
How fast is the API?	`reports/latency_report.json`
What are intended use and limitations?	`reports/model_card.md`

Question	Source of truth
How do I present it?	`docs/02_demo_script.md` and `scripts/demo.py`
What does testing cover?	`tests/`

Final takeaway

The strongest part of this Day 13 POC is not that logistic regression achieved a particular score on synthetic data. It is that the project turns a model into a governed decision-support system: target semantics are explicit, leakage is controlled, experiments are traceable, probability is calibrated, thresholds reflect operational constraints, uncertainty can abstain, subgroup behavior is visible, explanations carry caveats, predictions are versioned and auditable, and every major claim has an artifact or test behind it.

That is the core interview story: **build the smallest model that satisfies the measured business objective, then surround it with the data, evaluation, governance, serving, and operational evidence required to use it responsibly.**